

กลุ่มที่ 1 Smart Parking Iteration และ Recursion

ลานจอดรถมีทางเข้าและทางออกเพียงด้านเดียว รถที่เข้าก่อนจะอยู่ด้านใน ส่วนรถที่เข้าทีหลังจะอยู่ใกล้ทางออก หากรถที่อยู่ด้านในต้องการออก รถที่ขวางอยู่ต้องถูกนำออกชั่วคราว หลังจากนั้นนำรถเป้าหมายออกแล้วต้องนำรถที่ถูกย้ายกลับเข้าจอดโดยรักษาลำดับเดิม

1. การวิเคราะห์ปัญหา (Problem Analysis)

Input ของปัญหา

- ข้อมูลของรถแต่ละคันในลานจอด ซึ่งประกอบด้วย **ป้ายทะเบียน** (License Plate), **ชื่อเจ้าของ** (Owner Name), **เวลาเข้า** (Entry Time), และ**ประเภทรถ** (Vehicle Type)
- ข้อมูลเป้าหมาย ป้ายทะเบียนรถที่ต้องการนำออกจากลานจอด (Target License Plate)
- โครงสร้างลานจอดรถปัจจุบัน (Parking Stack)

Output ที่ต้องการ

- สถานะการดำเนินการ (Boolean) แจ้งว่าพบและนำรถเป้าหมายออกสำเร็จ (True) หรือไม่พบรถเป้าหมาย (False)
- สถานะของลานจอด (Parking Stack) รถเป้าหมายถูกนำออกไปแล้ว ในขณะที่รถคันอื่นๆ ที่ไม่ได้ถูกเลือก ยังคงอยู่ในลานจอดครบถ้วนและรักษา "ลำดับเดิม" ไว้ทุกประการ

ข้อจำกัดของปัญหา (Constraints)

- ลานจอดรถมีทางเข้าและทางออกเพียงด้านเดียว (ข้อจำกัดทางกายภาพนี้บังคับให้เข้าถึงรถได้เฉพาะคันที่อยู่ใกล้ทางออกที่สุดก่อน เหมือนโครงสร้างข้อมูลแบบ Stack หรือ LIFO - Last In, First Out)
- ห้ามทำรถที่จอดอยู่คันอื่นสูญหาย และห้ามสลับลำดับของรถคันอื่นเด็ดขาด
- การพัฒนาอัลกอริทึมต้องทำ 2 วิธี คือ Iteration (ใช้ Stack สำรอง) และ Recursion (ใช้ Call Stack)

Assumptions หรือข้อตกลงที่ใช้

- ป้ายทะเบียนรถ (License Plate) เป็นค่าเฉพาะ (Unique) ไม่มีรถสองคันที่ใช้ป้ายทะเบียนซ้ำกันในเวลาเดียวกัน
- ลานจอดรถมีพื้นที่เพียงพอสำหรับเก็บข้อมูลรถทุกคันโดยไม่เกิด Error ยกเว้นข้อจำกัดของระบบคอมพิวเตอร์ (Memory)

กรณีปกติ (Normal Case)

- รถเป้าหมายจอดอยู่บริเวณ "ตรงกลาง" ของลานจอด (ต้องย้ายรถคันอื่นบางส่วนออกไปพัก และย้ายกลับมาจอดที่เดิมได้อย่างถูกต้อง)

กรณีขอบเขต (Edge Cases)

- รถเป้าหมายจอดอยู่ "ด้านบนสุด" (ใกล้ทางออกที่สุด) ซึ่งถือเป็น Best Case
- รถเป้าหมายจอดอยู่ "ด้านล่างสุด" (ในสุด) ซึ่งถือเป็น Worst Case
- มีรถจอดอยู่ในลานเพียงคันเดียว
- ลานจอดรถมี "จำนวนรถมาก" (เพื่อทดสอบการจัดการหน่วยความจำและเวลา)
- ลานจอดรถว่างเปล่า (Stack ว่าง)

กรณีที่อาจทำให้โปรแกรมผิดพลาด (Error-Prone Cases)

- ป้ายทะเบียนเป้าหมายไม่มีอยู่ในลานจอดรถเลย (ไม่พบรถเป้าหมาย) โปรแกรมต้องค้นหาจนจบและคืนค่า False โดยไม่ค้างหรือพัง
- ป้อนค่า Input ป้ายทะเบียนเป็นค่าว่าง (Null หรือ Empty String)
- (สำหรับ Algorithm B) ความเสี่ยงที่จะเกิดเหตุการณ์ **Stack Overflow** เมื่อมีรถจำนวนมากในลานจอด แล้วใช้ Method แบบ Recursion ค้นหารถที่อยู่ลึกมากๆ

Operation ที่เกิดขึ้นบ่อย

- การเปรียบเทียบข้อมูล (Comparison) ตรวจสอบป้ายทะเบียนรถคันปัจจุบันกับเป้าหมายว่าตรงกันหรือไม่
- การใช้คำสั่ง pop() เพื่อนำรถออก และคำสั่ง push() เพื่อนำรถเข้าที่พัก (Temporary Stack) หรือนำกลับลานจอด

Operation ที่อาจใช้เวลามากที่สุด

- การนำรถทุกคันออกจากลานจอดแล้วนำกลับเข้าไปใหม่ (การทำ Pop และ Push จำนวนสูงสุด) ซึ่งจะเกิดขึ้นในกรณีที่ "รถเป้าหมายอยู่ล่างสุด" หรือ "ค้นหาจนหมดแล้วไม่พบรถเป้าหมาย" (Worst Case)

2. การออกแบบอัลกอริทึม

Algorithm A Iterative Method (แบบวนลูป)

ชื่ออัลกอริทึม RemoveCarIterative

แนวคิดหลัก ค้นหาเป้าหมายโดยการนำรถออกจากลานจอด (parkingStack) ทีละคัน หากไม่ใช่รถเป้าหมาย จะนำไปเก็บพักไว้ในลานจอดสำรอง (temporaryStack) เมื่อพบเป้าหมาย (หรือค้นหาจนหมด) จะนำรถจากลานจอดสำรองย้ายกลับเข้าลานจอดหลัก เพื่อรักษาลำดับเดิมของรถคันอื่นไว้

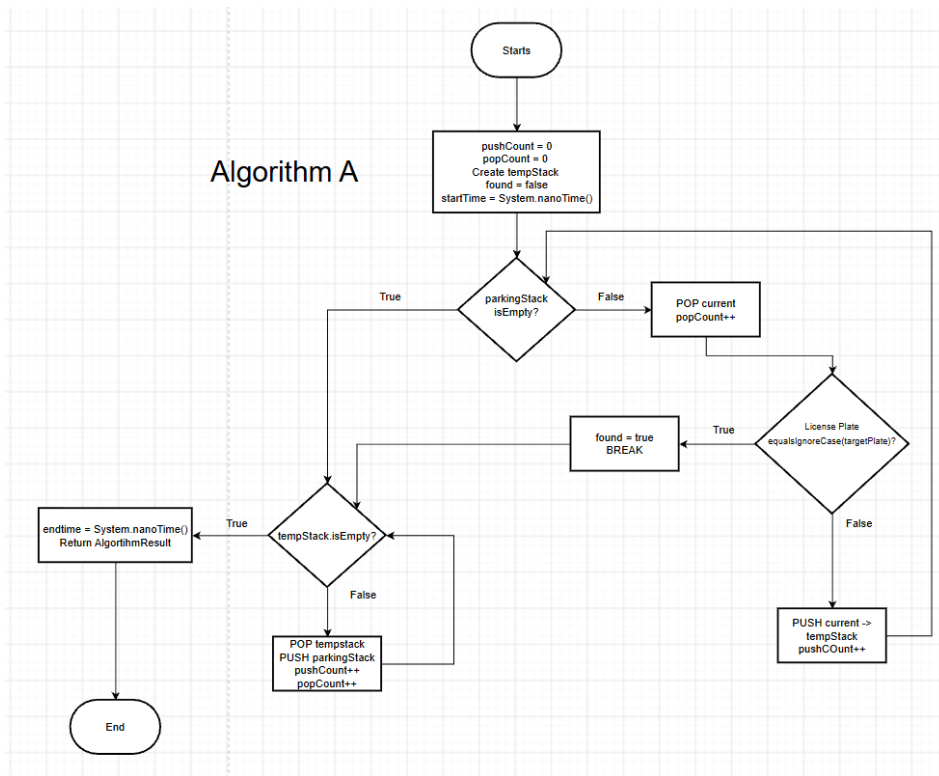
Input

- parkingStack Stack ที่เก็บข้อมูลรถในลานจอดปัจจุบัน
- targetPlate สตริง (String) ป้ายทะเบียนรถเป้าหมายที่ต้องการนำออก

Output

- true หากพบและนำรถเป้าหมายออกสำเร็จ
- false หากค้นหาจนหมดแล้วไม่พบรถเป้าหมาย

Pseudocode



ตัวอย่างการทำงาน

- Initial Stack บนสุด [C, B, A] ล่างสุด | เป้าหมาย = B
- Step 1 Pop C ออกมา ไม่ใช่เป้าหมาย -> Push C ใส่ temporaryStack (สถานะ temporary [C])
- Step 2 Pop B ออกมา คือเป้าหมาย -> found = true แล้วหยุดดู
- Step 3 นำข้อมูลจาก temporaryStack กลับเข้า parkingStack โดย Pop C ออกมา Push คีน
- Final Stack [C, A]

ข้อดี

- ปลอดภัยจากปัญหา Stack Overflow (หน่วยความจำเต็มจากการเรียกฟังก์ชันซ้อนกัน) เนื่องจากใช้หน่วยความจำ Heap ในการเก็บ temporaryStack
- ติดตามและอธิบายขั้นตอนการทำงาน (Trace) ได้ง่าย

ข้อจำกัด

- ต้องประกาศโครงสร้างข้อมูลเพิ่มเติม (temporaryStack) อย่างชัดเจน ทำให้โค้ดยาวกว่าแบบ Recursion เล็กน้อย

Time Complexity

- $O(n)$ ในกรณีแย่ที่สุด (ต้องนำรถออกทุกคัน)

Space Complexity

- Auxiliary Space $O(n)$ จากการสร้าง temporaryStack เพื่อรองรับรถทั้งหมดในกรณีที่แย่ที่สุด

Algorithm B Recursive Method (แบบเรียกตัวเอง)

ชื่ออัลกอริทึม RemoveCarRecursive

แนวคิดหลัก ใช้ Call Stack ของระบบแทน Stack สำรอง โดยดึงรถออกทีละคันแล้วเรียกใช้ฟังก์ชันตัวเอง (Recursive Call) เพื่อค้นหาอีกต่อไป หากพบรถเป้าหมายจะหยุดค้นหา เมื่อฟังก์ชันย้อนกลับ (Return) จะคืนรถที่ไม่ใช่เป้าหมายกลับคืนเข้าลานจอดหลักโดยอัตโนมัติ

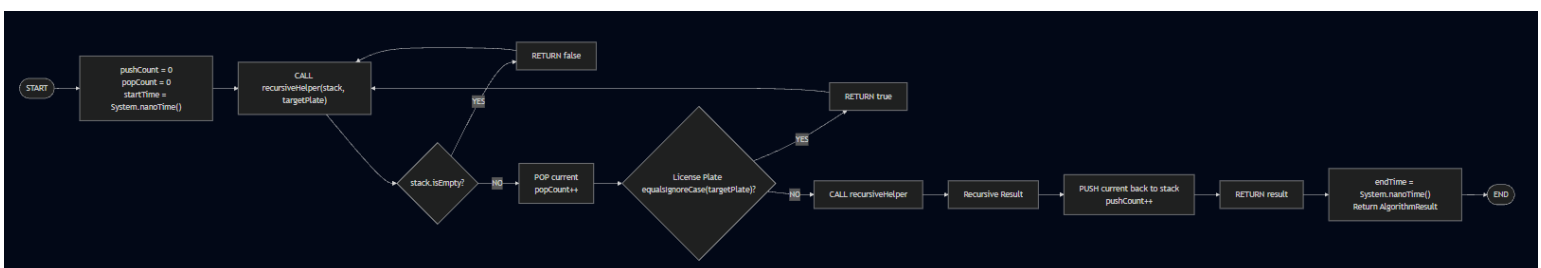
Input

- parkingStack Stack ที่เก็บข้อมูลรถในลานจอดปัจจุบัน
- targetPlate สตริง (String) ป้ายทะเบียนรถเป้าหมายที่ต้องการนำออก

Output

- **true** หากพบและนำรถเป้าหมายออกสำเร็จ
- **false** หากค้นหาจนหมด (Stack ว่าง) แล้วไม่พบรถเป้าหมาย

Pseudocode



ตัวอย่างการทำงาน

- **Initial Stack** บนสุด [C, B, A] ล่างสุด | เป้าหมาย = B
- **Call 1** Pop C -> ไม่ใช่ B -> เรียกตัวเอง Call 2
- **Call 2** Pop B -> คือเป้าหมาย -> คืนค่า true กลับไปที่ Call 1
- **กลับมาที่ Call 1** รับค่า true -> Push C กลับเข้า parkingStack -> คืนค่า true ออกจากฟังก์ชัน
- **Final Stack** [C, A]

ข้อดี

- โค้ดสั้น กระชับ และอ่านง่าย (Elegant)
- ไม่ต้องสร้างตัวแปรหรือโครงสร้างข้อมูล Stack สำรองแบบชัดเจน เพราะใช้ความสามารถของระบบจัดการให้

ข้อจำกัด

- มีความเสี่ยงสูงที่จะเกิด Stack Overflow หากมีจำนวนรถในลานจอดเยอะมากๆ (เช่น "n=50,000") เพราะระบบต้องจองพื้นที่ Call Stack ซ้อนกันลึกมาก
- มี Overhead จากการเรียกใช้งานฟังก์ชัน (Function Call) หลายครั้ง ทำให้ในทางปฏิบัติอาจใช้เวลาประมวลผลนานกว่าแบบ Loop เล็กน้อย

Time Complexity

- $O(n)$ ในกรณีแย่ที่สุด เพราะฟังก์ชันต้องเรียกตัวเองตามจำนวนชั้นของข้อมูล

Space Complexity

- Auxiliary Space $O(n)$ จากพื้นที่ Call Stack ของระบบที่ใช้จัดจำสถานะตัวแปรในแต่ละรอบการดึงรถออก

การแสดงตัวอย่างแบบ Step-by-step

กรณีปกติ (Normal Case - รถเป้าหมายจอดอยู่ตรงกลาง)

- ทะเบียน 04 (Somyod) - อยู่บนสุด/ใกล้ทางออก
- ทะเบียน 03 (Somying)
- ทะเบียน 02 (Somsak)
- ทะเบียน 01 (Somchai) - อยู่ล่างสุด

เป้าหมาย (Target) ต้องการนำรถทะเบียน 03 ออกจากลานจอด

- **Step 1** นำรถ 04 ออกจาก parkingStack -> ตรวจสอบแล้วไม่ใช่เป้าหมาย -> ย้ายไปจอดพักที่ temporaryStack

(สถานะลานจอดหลัก 03, 02, 01 | สถานะลานพัก 04)

- **Step 2** นำรถ 03 ออกจาก parkingStack -> ตรวจสอบพบว่าเป็นเป้าหมาย! -> ขับรถออกจากลานจอด (ไม่นำไปใส่ลานพัก)

(สถานะลานจอดหลัก 02, 01 | สถานะลานพัก 04)

- **Step 3** กระบวนการค้นหาจบลง เริ่มนำรถจากลานพักกลับเข้าที่เดิม โดยดึง 04 กลับเข้า parkingStack

(สถานะลานจอดหลัก 04, 02, 01 | สถานะลานพัก ว่างเปล่า)

กรณีเลวร้ายที่สุด (Worst Case - รถเป้าหมายจอดอยู่ลึกที่สุด) Algorithm B

- ทะเบียน 04 (Somyod) - อยู่บนสุด/ใกล้ทางออก
- ทะเบียน 03 (Somying)
- ทะเบียน 02 (Somsak)
- ทะเบียน 01 (Somchai) - อยู่ล่างสุด

เป้าหมาย (Target) ต้องการนำรถทะเบียน 01 ออกจากลานจอด

- **Step 1** (Call 1) pop() 04 ออก และเรียกฟังก์ชันตัวเองซ้ำ (Recursive Call)

(สถานะลานจอดหลัก 03, 02, 01 | สถานะลานพัก 04)

- **Step 2 (Call 2) pop() 03** ออก และเรียกฟังก์ชันตัวเองซ้ำ
(สถานะลานจอดหลัก 02, 01 | สถานะลานพัก 04, 03)
- **Step 3 (Call 3) pop() 02** ออก และเรียกฟังก์ชันตัวเองซ้ำ
(สถานะลานจอดหลัก 01 | สถานะลานพัก 04, 03, 02)
- **Step 4 (Call 4) pop() 01** ตรวจสอบแล้ว "พบเป้าหมาย" คืนค่า true และไม่นำรถคันนี้กลับเข้า Stack (Base Case)
(สถานะลานจอดหลัก **ว่างเปล่า** | สถานะลานพัก 04, 03, 02)
- **Step 5 (Backtracking)** ระบบจะย้อนกลับตามลำดับ Call stack เพื่อนำรถที่ขวางอยู่วางกลับลงไป

Backtrack 3 push 02 กลับเข้า Stack

(สถานะลานจอดหลัก 02 | สถานะลานพัก 04, 03)

Backtrack 2 push 03 กลับเข้า Stack

(สถานะลานจอดหลัก 03, 02 | สถานะลานพัก 04)

Backtrack 1 push 04 กลับเข้า Stack

(สถานะลานจอดหลัก 04, 03, 02 | สถานะลานพัก **ว่างเปล่า**)

3. การอธิบายความถูกต้อง (Correctness Argument)

Algorithm A Iterative Method โดยใช้วิธี Loop Invariant

ก่อนเริ่มแต่ละรอบของ Loop รถทุกคันที่ถูกนำออกจาก Parking Stack และยังไม่ใช้รถเป้าหมายจะอยู่ใน Temporary Stack โดยไม่มีรถสูญหาย และรถที่ยังอยู่ใน Parking Stack ยังคงลำดับเดิม

1. ก่อนเริ่มอัลกอริทึม ข้อมูลอยู่ในสภาพใด (Initialization)

- ก่อนเข้าสู่ลูปการค้นหา temporaryStack จะเป็น Stack ว่างเปล่า
- รถทุกคันยังคงจอดอยู่ใน parkingStack อย่างครบถ้วนและอยู่ในลำดับเดิมสมบูรณ์แบบ
- ยังไม่มีการดำเนินการย้ายรถใดๆ คุณสมบัติ Invariant จึงเป็นจริงตั้งแต่เริ่มต้น

2. ระหว่างทำงาน มีคุณสมบัติใดที่ยังคงเป็นจริง (Maintenance)

- ในแต่ละรอบของลูป WHILE รถที่อยู่บนสุดจะถูก pop() ออกจาก parkingStack
- หากรถคันนั้น ไม่ใช่เป้าหมาย มันจะถูก push() เข้าไปเก็บใน temporaryStack ทันที
- การทำเช่นนี้รับประกันได้ว่า จะไม่มีรถคันใดสูญหายไปจากระบบ (ถ้ารถไม่อยู่ในลานจอด ก็ต้องอยู่ในลานพัก) และรถที่ยังรอการตรวจสอบใน parkingStack ก็ยังคงเรียงลำดับเหมือนเดิมทุกประการ คุณสมบัติ Invariant จึงได้รับการรักษาไว้ในทุกๆ รอบของลูป

3. เมื่ออัลกอริทึมหยุด ผลลัพธ์ถูกต้องตามเงื่อนไขหรือไม่

- อัลกอริทึมจะหยุดค้นหาเมื่อพบรถเป้าหมาย (ทำคำสั่ง break) หรือเมื่อตรวจสอบรถจนหมดลานจอด (parkingStack ว่างเปล่า)
- หลังจากนั้น โปรแกรมจะเข้าสู่ลูปที่สอง เพื่อนำรถทั้งหมดใน temporaryStack ย้ายกลับเข้า parkingStack
- ผลลัพธ์สุดท้ายคือ รถเป้าหมายถูกนำออกไปสำเร็จ (หากมี) และรถคันอื่นๆ ถูกนำกลับเข้าที่เดิมอย่างถูกต้องตามเงื่อนไขของโจทย์

4. ข้อมูลที่ไม่เกี่ยวข้องสูญหายหรือเปลี่ยนลำดับหรือไม่

- ไม่สูญหายและไม่เปลี่ยนลำดับ ลักษณะการทำงานของ Stack แบบ LIFO (Last In, First Out)
- เมื่อเราย้ายรถคันที่ไม่เกี่ยวข้องไปที่ temporaryStack ลำดับของรถจะถูก "กลับหัว" ชั่วคราว
- แต่เมื่อเราทำลูป push() กลับคืนจาก temporaryStack สู่ parkingStack ลำดับจะถูก "กลับหัวอีกครั้ง" ส่งผลให้ลำดับการจอดรถกลับมาเป็นเหมือนเดิม 100% โดยไม่มีรถคันใดสูญหาย

5. อัลกอริทึมหยุดทำงานได้เสมอหรือไม่ (Termination)

- อัลกอริทึมหยุดทำงานได้เสมอ (Guaranteed Termination)

- เนื่องจากในแต่ละรอบของลูป WHILE แรก จะมีการทำคำสั่ง `parkingStack.pop()` เสมอ ซึ่งเป็นการลดจำนวนข้อมูลใน Stack ลงทีละ 1
- ในกรณีที่แย่ที่สุด (Worst Case) หากไม่พบรถเป้าหมายเลย จำนวนรถใน `parkingStack` จะลดลงจนเป็น 0 (`isEmpty() == true`) ทำให้เงื่อนไขลูปเป็นเท็จ และหลุดออกจากลูปได้อย่างแน่นอน โปรแกรมจึงไม่มีทางเกิด Infinity Loop

4. การวิเคราะห์ประสิทธิภาพ

Algorithm	Best Case	Average Case	Worst Case	Auxiliary Space
Algorithm A	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Algorithm B	$O(1)$	$O(n)$	$O(n)$	$O(n)$

1. Time Complexity (ความซับซ้อนของเวลา)

- Best-case Time Complexity $O(1)$

เกิดขึ้นเมื่อรถเป้าหมายจอดอยู่ตำแหน่ง "บนสุด" (Top of Stack) ของลานจอดพอดี ระบบทำการดึงรถออก (`pop()`) เพียง 1 ครั้ง ตรวจสอบพบว่าเป็นเป้าหมาย และหยุดการทำงานทันทีโดยไม่ต้องย้ายรถคันอื่นเลย ฟังก์ชันจึงใช้เวลาคงที่เสมอ ไม่ว่าจะมียอดรถในลานจอดกี่คันก็ตาม

- Worst-case Time Complexity $O(n)$

เกิดขึ้นเมื่อรถเป้าหมายจอดอยู่ "ด้านในสุด" (Bottom) หรือ "ไม่มีรถเป้าหมายในลานจอดเลย" ระบบจะต้องทำการ `pop()` รถออกจนหมดจำนวน n คัน และหลังจากนั้นต้องวนลูปหรือย้อนฟังก์ชันกลับเพื่อ `push()` รถทั้งหมดกลับเข้าลานจอดหลักอีก n ครั้ง (หรือ $n - 1$ ครั้ง) จำนวน Operation จึงแปรผันตรงกับจำนวนข้อมูล n

- Average-case Time Complexity $O(n)$

หากรถเป้าหมายอยู่ตรงกลางลานจอด ระบบต้องดึงรถออกประมาณ $n/2$ คัน และดันกลับ $n/2$ คัน ในทางคณิตศาสตร์ของการคำนวณ Big-O เมื่อตัดค่าคงที่ ($1/2$) ออกไป จะยังคงถือว่าความซับซ้อนของเวลาเติบโตในระดับ $O(n)$

2. Space Complexity (ความซับซ้อนของพื้นที่หน่วยความจำ)

- Auxiliary Space Complexity (พื้นที่เสริม) $O(n)$
- Algorithm A (Iterative) ต้องใช้หน่วยความจำในการสร้าง temporaryStack เพื่อพักรถที่ถูกย้ายออกชั่วคราว ในกรณีแย่ที่สุด (Worst Case) ต้องย้ายรถไปพักถึง n คัน จึงต้องใช้พื้นที่เสริมระดับ $O(n)$
- Algorithm B (Recursive) แม้จะไม่ได้ประกาศ Stack สำรองอย่างชัดเจน แต่การเรียกฟังก์ชันตัวเองซ้อนกัน จะต้องใช้พื้นที่ Call Stack ของระบบปฏิบัติการเพื่อจดจำตัวแปร currentCar ในแต่ละชั้น ในกรณีแย่ที่สุดฟังก์ชันต้องเรียกซ้อนกันลึก n ชั้น จึงใช้พื้นที่เสริม $O(n)$
- Total Space Complexity (พื้นที่รวม) $O(n)$
พื้นที่รวมคำนวณจาก ข้อมูล Input เริ่มต้น (รถใน parkingStack จำนวน n คัน ซึ่งมีขนาด $O(n)$) รวมกับ Auxiliary Space ($O(n)$) ผลรวมคือ $O(n) + O(n) = O(2n)$ เมื่อตัดค่าคงที่ออก จะได้ Total Space Complexity เป็น $O(n)$

Test Cases บังคับ

- 1. รถเป้าหมายอยู่ด้านบนสุด
- 2. รถเป้าหมายอยู่ตรงกลาง
- 3. รถเป้าหมายอยู่ด้านล่างสุด
- 4. ไม่พบรถเป้าหมาย
- 5. Stack ว่าง
- 6. มีรถเพียงคันเดียว
- 7. มีรถจำนวนมาก
- 8. เปรียบเทียบ Iteration กับ Recursion

Test ID	ประเภทการทดสอบ	Input / Action	Expected Result (ผลลัพธ์ที่คาดหวัง)	Actual Result (ผลลัพธ์ที่ได้อ้างอิง)	Status
TC01	Best Case	ค้นหาเป้าหมายที่อยู่ด้านบนสุด (เข้าจอดล่าสุด)	ระบบคืนค่า True (พบรถ) นำรถออกสำเร็จโดยไม่ต้องย้ายรถคันอื่น	ระบบคืนค่า True ไม่มีการย้ายรถคันอื่น	Pass
TC02	Normal Case	ค้นหาเป้าหมายที่อยู่ตรงกลาง	ระบบคืนค่า True ย้ายรถที่ขวางไปพัก และย้ายกลับมากในลำดับเดิมได้อย่างถูกต้อง	ระบบคืนค่า True รถคันอื่นอยู่ในลำดับเดิม	Pass
TC03	Worst Case	ค้นหาเป้าหมายที่อยู่ด้านล่างสุด (เข้าจอดคันแรก)	ระบบคืนค่า True ย้ายรถทุกคัน (ยกเว้นเป้าหมาย) ไปพัก และนำกลับเข้าลานจอดครบถ้วน	ระบบคืนค่า True รถคันอื่นอยู่ในลำดับเดิม	Pass
TC04	Target Not Found	ค้นหาป้ายทะเบียนที่ไม่มีในลานจอด	ระบบคืนค่า False (ไม่พบรถ) โปรแกรมไม่ค้าง และรถทุกคันกลับสู่ลำดับเดิม	ระบบคืนค่า False รถทุกคันอยู่ในลำดับเดิม	Pass
TC05	Empty Stack	ค้นหาเป้าหมายในขณะที่ "Stack ว่าง"	ระบบคืนค่า False แสดงข้อความแจ้งเตือนว่าลานจอดว่างเปล่า	แสดงข้อความเตือนลานจอดว่าง ระบบไม่เกิด Error	Pass
TC06	Boundary Case	มีรถเพียงคันเดียวในลานจอด และค้นหาคันนั้น	ระบบคืนค่า True ลานจอดกลายเป็นสถานะว่างเปล่าทันที	ระบบคืนค่า True ลานจอดไม่มีรถตกค้าง	Pass
TC07	Invalid Input	ป้อนป้ายทะเบียนเป็นคำว่าง (Empty String)	ระบบแสดงข้อความเตือนให้กรอกข้อมูล ไม่ทำการดึงข้อมูลออกจาก Stack	แสดงข้อความ "[Error] ทะเบียนรถห้ามเป็นคำว่าง!"	Pass
TC08	Stress Test / System Specific	มีรถจำนวนมาก (เช่น 10,000 คัน) และเลือกใช้ Recursion (Algorithm B)	ระบบอาจเกิด StackOverflowError เนื่องจากข้อจำกัดของ Call Stack	เกิด StackOverflowError ตามที่วิเคราะห์ไว้ใน Space Complexity	Pass

5. การทดลองประสิทธิภาพ

ให้ทดสอบข้อมูลอย่างน้อย 4 ขนาด เช่น

n = 100

n = 1,000

n = 10,000

n = 50,000

แต่ละขนาดต้องทดลองอย่างน้อย 5 รอบ และรายงานค่าเฉลี่ย

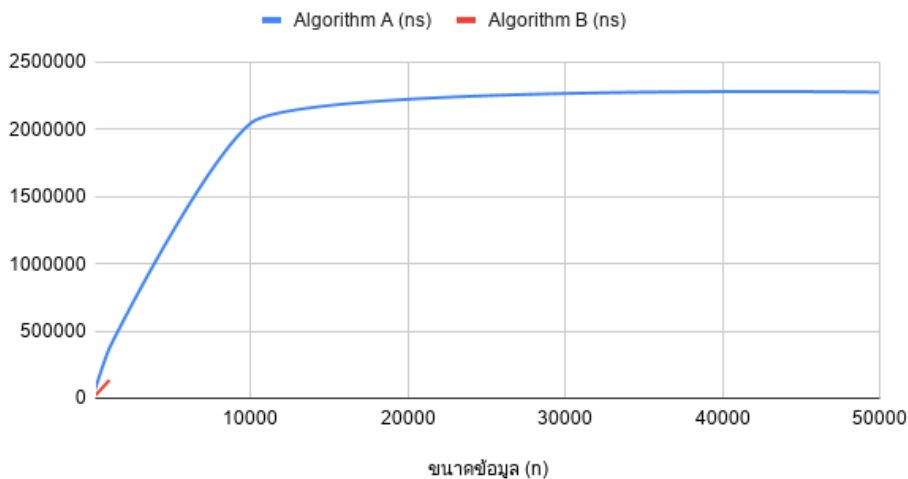
n	Algo	Avg Time(ns)	Push	Pop	Compare	Loop	Moved	Space(Max)
100	A	69920	200	200	100	200	200	100
100	B	19820	100	100	100	101	200	101
1000	A	372440	2000	2000	1000	2000	2000	1000
1000	B	134480	1000	1000	1000	1001	2000	1001
10000	A	2045800	20000	20000	10000	20000	20000	10000
10000	B	STACK OVERFLOW	-	-	-	-	-	-
50000	A	2274780	100000	100000	50000	100000	100000	50000
50000	B	STACK OVERFLOW	-	-	-	-	-	-

ตารางผลการทดลอง

n	Algorithm	Average Time (ns)	Push	Pop	Comparisons	Loop	Moved	Space(Max)
100	A (Iterative)	69,920	200	200	100	200	200	100
100	B (Recursive)	19,820	100	100	100	101	200	101
1,000	A (Iterative)	372,440	2,000	2,000	1,000	2,000	2,000	1,000
1,000	B (Recursive)	134,480	1,000	1,000	1,000	1,001	2,000	1,001
10,000	A (Iterative)	2,045,800	20,000	20,000	10,000	20,000	20,000	10,000
10,000	B (Recursive)	STACK OVERFLOW	-	-	-	-	-	-
50,000	A (Iterative)	2,274,780	100,000	100,000	50,000	100,000	100,000	50,000
50,000	B (Recursive)	STACK OVERFLOW	-	-	-	-	-	-

กราฟผลการทดลอง

กราฟผลการทดลอง Algorithm A (ns) & Algorithm B (ns)



อภิปรายผลการทดลอง

1. Algorithm B (เส้นสีแดง - Recursive Method)

- ผลจากกราฟ กราฟเส้นสีแดงแสดงผลแค่ช่วงข้อมูลขนาดเล็ก ($n = 100$ ถึง $n = 1,000$) และขาดหายไปเมื่อข้อมูลมีขนาด 10,000 ขึ้นไป
- ความสอดคล้องกับ Big-O สอดคล้องกับทฤษฎี Space Complexity ที่วิเคราะห์ไว้ว่าเป็น $O(n)$ เนื่องจากเมื่อ n มีขนาดใหญ่ การเรียกฟังก์ชันตัวเองซ้ำๆ จะทำให้เปลืองพื้นที่ Call Stack ในระบบจนทะลุขีดจำกัด ทำให้เกิด STACK OVERFLOW โปรแกรมจึงไม่สามารถทำงานต่อได้

2. Algorithm A (เส้นสีน้ำเงิน - Iterative Method)

- ผลจากกราฟ กราฟเส้นสีน้ำเงินสามารถทำงานได้จนจบถึง $n = 50,000$ แสดงให้เห็นถึงความเสถียรของการใช้ Temporary Stack (Heap Memory) ที่รองรับข้อมูลได้มากกว่า
- ความสอดคล้องกับ Big-O ตามทฤษฎี Time Complexity ของ Algorithm A คือ $O(n)$ ซึ่งกราฟควรจะเป็นเส้นตรงทแยงมุมขึ้น (Linear) แต่จากกราฟจะเห็นว่าช่วงแรก ($n = 0$ ถึง $n = 10,000$) เวลาเพิ่มขึ้นเป็นเส้นตรงตามทฤษฎี แต่หลังจากนั้นกราฟเริ่มโค้งและแบนราบลง (เวลาเพิ่มขึ้นน้อยมากแม้ข้อมูลจะเพิ่มเป็น 50,000)

คำถามวิเคราะห์

1. Best Case และ Worst Case เกิดขึ้นเมื่อใด

- **Best Case** เกิดขึ้นเมื่อ "รถเป้าหมายจอดอยู่ด้านบนสุดของลานจอด (ใกล้ทางออกที่สุด)" เพราะโปรแกรมจะทำการดึงข้อมูล (Pop) เพียง 1 ครั้ง ตรวจสอบพบว่าเป็นเป้าหมาย และทำงานเสร็จสิ้นทันทีโดยไม่ต้องย้ายรถคันอื่นเลย
- **Worst Case** เกิดขึ้นเมื่อ "รถเป้าหมายจอดอยู่ด้านล่างสุดของลานจอด (ลึกที่สุด)" หรือ "ไม่พบรถเป้าหมายในลานจอดเลย" เพราะโปรแกรมจะต้องดึงรถทุกคันออกไปพักจนหมดลานจอด แล้วต้องนำรถทุกคันดันกลับเข้าลานจอดเดิม ทำให้เกิด Operation จำนวนมากที่สุด

2. หากรถเป้าหมายอยู่ลำดับที่ k จากด้านบน ต้องใช้ Pop กี่ครั้ง

- หากนับรวม ขั้นตอนการนำรถกลับเข้าลานจอด จะต้อง Pop รถที่ไปพักไว้ออกมาอีก $k - 1$ ครั้ง (เพราะรถเป้าหมายคันที่ k ถูกนำออกไปแล้ว) รวมทั้งหมดในระบบจะต้องใช้คำสั่ง Pop ทั้งหมด $2k - 1$ ครั้ง

3. เหตุใด Iteration และ Recursion มี Time Complexity ใกล้เคียงกัน

- เนื่องจากอัลกอริทึมทั้งสองวิธีมีพฤติกรรมในการเข้าถึงข้อมูลแบบเดียวกัน คือต้องดึงข้อมูลรถออกจาก Stack ที่ละคันจากบนลงล่างเพื่อตรวจสอบ หากมีรถวางอยู่จำนวน n คัน ทั้งวิธีวนลูป (Iteration) และวิธีเรียกตัวเอง (Recursion) ก็ต้องทำการดึงเข้า-ออก n รอบเท่าๆ กัน ทำให้ระยะเวลาที่ใช้ในการประมวลผลเติบโตเป็นสัดส่วนตรงกับจำนวนข้อมูล ส่งผลให้มี Time Complexity ในกรณี Worst Case เท่ากันคือ $O(n)$

4. เหตุใด Recursion ใช้ Auxiliary Space เพิ่มเติม

- แม้วิธี Recursion จะไม่ได้ประกาศตัวแปร temporaryStack ขึ้นมาพักข้อมูลจริงๆ เหมือนวิธี Iteration แต่ทุกครั้งที่ฟังก์ชันทำการเรียกตัวเอง (Recursive Call) ระบบปฏิบัติการ (OS/JVM) จำเป็นจะต้องจองพื้นที่หน่วยความจำที่เรียกว่า "Call Stack" เพื่อเก็บสถานะของตัวแปร (เช่น รถที่เพิ่ง Pop ออกมา) ของฟังก์ชันในรอบนั้นๆ เอาไว้ หากฟังก์ชันเรียกซ้อนกันลึก n ชั้น ก็จะใช้พื้นที่หน่วยความจำระดับ $O(n)$ ทำให้สิ้นเปลือง Auxiliary Space และมีความเสี่ยงที่จะเกิด Stack Overflow หากข้อมูลมีปริมาณมาก

5. Loop Invariant รับประกันความถูกต้องอย่างไร

- Loop Invariant เป็นเครื่องมือทางตรรกศาสตร์ที่ใช้พิสูจน์ความถูกต้องของโปรแกรม โดยบังคับเงื่อนไขว่า "ก่อนเริ่มลูป, ระหว่างทำลูป, และหลังจบลูป รถที่ไม่ใช่เป้าหมายจะต้องถูกเก็บอย่างปลอดภัยโดยไม่สูญหาย และเมื่อนำกลับต้องคงลำดับเดิมเสมอ"